

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250272024

Kind Code

A1

Publication Date

August 28, 2025

Inventor(s)

Achkinazi; Igor

STORAGE SYSTEM WITH DYNAMIC ADJUSTMENT OF NETWORK RESOURCES

Abstract

An apparatus comprises at least one processing device that includes a processor coupled to a memory. The at least one processing device is configured to receive in a storage system from a host device a command specifying an initial number of input-output (IO) queues to be configured for a given target of the storage system, to configure in the storage system the initial number of IO queues for the given target responsive to the received command, and to dynamically adjust in the storage system the number of IO queues configured for the given target based at least in part on an amount of IO operations received by the storage system for the given target. Dynamically adjusting the number of IO queues illustratively comprises increasing the number of IO queues relative to the initial number of IO queues based at least in part on a utilization level of the IO queues.

Inventors: Achkinazi; Igor (Northborough, MA)

Applicant: Dell Products L.P. (Round Rock, TX)

Family ID: 1000007739373

Appl. No.: 18/586843

Filed: February 26, 2024

Publication Classification

Int. Cl.: G06F3/06 (20060101)

U.S. Cl.:

CPC G06F3/0659 (20130101); G06F3/0613 (20130101); G06F3/067 (20130101);

Background/Summary

FIELD

[0001] The field relates generally to information processing systems, and more particularly to storage in information processing systems.

BACKGROUND

[0002] Information processing systems often include distributed storage systems comprising multiple storage nodes. These distributed storage systems may be dynamically reconfigurable under software control in order to adapt the number and type of storage nodes and the corresponding system storage capacity as needed, in an arrangement commonly referred to as a software-defined storage system. For example, in a typical software-defined storage system, storage capacities of multiple distributed storage nodes are pooled together into one or more storage pools. For applications running on a host that utilizes the software-defined storage system, such a storage system provides a logical storage object view to allow a given application to store and access data, without the application being aware that the data is being dynamically distributed among different storage nodes. In these and other software-defined storage system arrangements, it can be unduly difficult to configure an appropriate amount of network resources for targets of respective storage nodes, particularly when using advanced storage access protocols such as Non-Volatile Memory Express (NVMe) over Fabrics, also referred to as NVMe-oF, or NVMe over Transmission Control Protocol (TCP), also referred to as NVMe/TCP. For example, conventional approaches are susceptible to overallocation of network resources, which can undermine storage system performance.

SUMMARY

[0003] Illustrative embodiments disclosed herein provide techniques for dynamic adjustment of network resources, in a software-defined storage system or other type of storage system, such as numbers of input-output (IO) queues and associated TCP connections configured for each of one or more NVMe targets of the storage system. Such techniques advantageously facilitate the usage of advanced storage access protocols such as NVMe-oF or NVMe/TCP while avoiding the above-described drawbacks of conventional practice. For example, some embodiments can substantially improve storage system performance by dynamically adjusting network resources, such as the number of IO queues and corresponding TCP connections for NVMe targets of the storage system, responsive to actual usage of the network resources over time.

[0004] Although some embodiments are described herein in the context of implementing an NVMe-oF or NVMe/TCP access protocol in a software-defined storage system, it is to be appreciated that other embodiments can be implemented in other types of distributed storage systems using other storage access protocols.

[0005] In addition, the disclosed techniques can be implemented in other embodiments in stand-alone storage arrays or other types of storage systems that are not distributed across multiple storage nodes. Accordingly, the disclosed techniques are applicable to a wide variety of different types of storage systems.

[0006] In one embodiment, an apparatus comprises at least one processing device that includes a processor coupled to a memory. The at least one processing device is configured to receive in a storage system from a host device a command specifying an initial number of IO queues to be configured for a given target of the storage system, to configure in the storage system the initial number of IO queues for the given target responsive to the received command, and to dynamically adjust in the storage system the number of IO queues configured for the given target based at least in part on an amount of IO operations received by the storage system for the given target.

[0007] The at least one processing device illustratively comprises at least a portion of the storage system.

[0008] In some embodiments, each of the IO queues configured in the storage system for the given target is associated with a corresponding different TCP connection between the host device and the

target.

[0009] The given target may comprise at least one NVMe controller of the storage system, although other types of targets can be used.

[0010] In some embodiments, the command comprises a Set Features command having a Feature Identifier specifying the initial number of IO queues for the given target and is received in the given target of the storage system from an initiator of the host device. Other types of commands can be used in other embodiments to establish the initial number of IO queues to be configured for the given target in the storage system.

[0011] In some embodiments, dynamically adjusting in the storage system the number of IO queues configured for the given target based at least in part on an amount of IO operations received by the storage system for the given target illustratively comprises increasing the number of IO queues relative to the initial number of IO queues based at least in part on a utilization level of the IO queues. The utilization level of the IO queues is illustratively determined collectively across all currently-configured IO queues for the given target for each of a plurality of monitoring intervals.

[0012] In some embodiments, increasing the number of IO queues relative to the initial number of IO queues based at least in part on the utilization level of the IO queues comprises adding one or more IO queues and respective associated TCP connections to a total number of IO queues and respective associated TCP connections previously configured for the given target, responsive to the utilization level of the IO queues exceeding a designated threshold utilization level. The designated threshold utilization level may be expressed at least in part as a particular percentage of full utilization of the IO queues for the given target.

[0013] Additionally or alternatively, dynamically adjusting in the storage system the number of IO queues configured for the given target based at least in part on an amount of IO operations received by the storage system for the given target comprises varying the number of IO queues configured for the given target within a range from the initial number of IO queues for the given target to a specified maximum number of IO queues for the given target.

[0014] As indicated above, the storage system in some embodiments illustratively comprises a distributed storage system that includes a plurality of storage nodes. The distributed storage system may more particularly comprise, for example, a software-defined storage system in which the storage nodes illustratively comprise respective software-defined storage server nodes of the software-defined storage system. Again, these and other features mentioned above are examples only, and should not be viewed as limiting in any way.

[0015] These and other illustrative embodiments include, without limitation, apparatus, systems, methods and processor-readable storage media.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0016] FIG. 1 is a block diagram of an information processing system incorporating functionality for dynamic adjustment of network resources in a distributed storage system in an illustrative embodiment.

[0017] FIG. 2 is a flow diagram of a process for dynamic adjustment of network resources in a distributed storage system in an illustrative embodiment.

[0018] FIG. 3A shows another example of an information processing system incorporating functionality for dynamic adjustment of network resources in a software-defined storage system in an illustrative embodiment. FIG. 3B shows example pseudocode for dynamic adjustment of network resources in such an embodiment. These figures are collectively referred to herein as FIG. 3.

[0019] FIG. 4 shows another example of an information processing system incorporating

functionality for dynamic adjustment of network resources in an illustrative embodiment.

[0020] FIG. 5 shows a further example of an information processing system incorporating functionality for dynamic adjustment of network resources in an illustrative embodiment.

[0021] FIGS. 6 and 7 show examples of processing platforms that may be utilized to implement at least a portion of an information processing system in illustrative embodiments.

DETAILED DESCRIPTION

[0022] Illustrative embodiments will be described herein with reference to exemplary information processing systems and associated computers, servers, storage devices and other processing devices. It is to be appreciated, however, that these and other embodiments are not restricted to the particular illustrative system and device configurations shown. Accordingly, the term “information processing system” as used herein is intended to be broadly construed, so as to encompass, for example, processing systems comprising cloud computing and storage systems, as well as other types of processing systems comprising various combinations of physical and virtual processing resources. An information processing system may therefore comprise, for example, at least one data center or other cloud-based system that includes one or more clouds hosting multiple tenants that share cloud resources, as well as other types of systems comprising a combination of cloud and edge infrastructure. Numerous different types of enterprise computing and storage systems are also encompassed by the term “information processing system” as that term is broadly used herein.

[0023] FIG. 1 shows an information processing system **100** configured in accordance with an illustrative embodiment. The information processing system **100** comprises a plurality of hosts **101-1, 101-2, . . . 101-N**, collectively referred to herein as hosts **101**, and a distributed storage system **102** shared by the hosts **101**. The hosts **101** and distributed storage system **102** in this embodiment are configured to communicate with one another via a network **104** that illustratively utilizes protocols such as Transmission Control Protocol (TCP) and Internet Protocol (IP), and is therefore referred to herein as a TCP/IP network, although it is to be appreciated that the network **104** can operate using additional or alternative protocols. In some embodiments, the network **104** comprises a storage area network (SAN) that includes one or more Fibre Channel (FC) switches, Ethernet switches or other types of switch fabrics.

[0024] It should be noted that the term “host” as used herein is intended to be broadly construed, so as to encompass, for example, a host device or a host system, each of which may comprise multiple distinct devices of various types. A host in some embodiments can comprise, for example, at least one server, as well as additional or alternative types and arrangements of processing devices.

[0025] The distributed storage system **102** more particularly comprises a plurality of storage nodes **105-1, 105-2, . . . 105-M**, collectively referred to herein as storage nodes **105**. The values N and M in this embodiment denote arbitrary integer values that in the figure are illustrated as being greater than or equal to three, although other values such as N=1, N=2, M=1 or M=2 can be used in other embodiments.

[0026] The storage nodes **105** collectively form the distributed storage system **102**, which is just one possible example of what is generally referred to herein as a “distributed storage system.” Other distributed storage systems can include different numbers and arrangements of storage nodes, and possibly one or more additional components. For example, as indicated above, a distributed storage system in some embodiments may include only first and second storage nodes, corresponding to an M=2 embodiment. Some embodiments can configure a distributed storage system to include additional components in the form of a system manager implemented using one or more additional nodes.

[0027] In some embodiments, the distributed storage system **102** provides a logical address space that is divided among the storage nodes **105**, such that different ones of the storage nodes **105** store the data for respective different portions of the logical address space. Accordingly, in these and other similar distributed storage system arrangements, different ones of the storage nodes **105** have responsibility for different portions of the logical address space. For a given logical storage

volume, logical blocks of that logical storage volume are illustratively distributed across the storage nodes **105**.

[0028] Other types of distributed storage systems can be used in other embodiments. For example, distributed storage system **102** can comprise multiple distinct storage arrays, such as a production storage array and a backup storage array, possibly deployed at different locations. Accordingly, in some embodiments, one or more of the storage nodes **105** may each be viewed as comprising at least a portion of a separate storage array with its own logical identifier (e.g., address) space. Alternatively, the storage nodes **105** can be viewed as collectively comprising one or more storage arrays. The term “storage node” as used herein is therefore intended to be broadly construed.

[0029] In some embodiments, the distributed storage system **102** comprises a software-defined storage system and the storage nodes **105** comprise respective software-defined storage server nodes of the software-defined storage system, such nodes also being referred to herein as SDS server nodes, where SDS denotes software-defined storage. Accordingly, the number and types of storage nodes **105** can be dynamically expanded or contracted under software control in some embodiments. Examples of such software-defined storage systems will be described in more detail below in conjunction with FIG. 3.

[0030] It is to be appreciated, however, that techniques disclosed herein can be implemented in other embodiments in stand-alone storage arrays or other types of storage systems that are not distributed across multiple storage nodes. The disclosed techniques are therefore applicable to a wide variety of different types of storage systems. The distributed storage system **102** is just one illustrative example.

[0031] In the distributed storage system **102**, each of the storage nodes **105** is illustratively configured to interact with one or more of the hosts **101**. The hosts **101** illustratively comprise servers or other types of computers of an enterprise computer system, cloud-based computer system or other arrangement of multiple compute nodes, each associated with one or more system users.

[0032] The hosts **101** in some embodiments illustratively provide compute services such as execution of one or more applications on behalf of each of one or more users associated with respective ones of the hosts **101**. Such applications illustratively generate input-output (IO) operations that are processed by a corresponding one of the storage nodes **105**. The term “input-output” as used herein refers to at least one of input and output. For example, IO operations may comprise write requests and/or read requests directed to logical addresses of a particular logical storage volume of one or more of the storage nodes **105**. These and other types of IO operations are also generally referred to herein as IO requests.

[0033] The IO operations that are currently being processed in the distributed storage system **102** in some embodiments are referred to herein as outstanding IOs that have been admitted by the storage nodes **105** to further processing within the system **100**. The storage nodes **105** are illustratively configured to queue IO operations arriving from one or more of the hosts **101** in one or more sets of IO queues. In some embodiments, each of the storage nodes **105** comprises one or more NVMe targets or other types of targets of the distributed storage system **102**, and each such target is configured with a plurality of IO queues. Each such IO queue may have a corresponding TCP connection or other type of network connection with one or more of the hosts **101**. Such IO queues and network connections are considered examples of “network resources” as that term is broadly used herein.

[0034] The storage nodes **105** illustratively comprise respective processing devices of one or more processing platforms. For example, the storage nodes **105** can each comprise one or more processing devices each having a processor and a memory, possibly implementing virtual machines and/or containers, although numerous other configurations are possible.

[0035] The storage nodes **105** can additionally or alternatively be part of cloud infrastructure, such as a cloud-based system implementing Storage-as-a-Service (STaaS) functionality.

[0036] The storage nodes **105** may be implemented on a common processing platform, or on separate processing platforms. In the case of separate processing platforms, there may be a single storage node per processing platform or multiple storage nodes per processing platform.

[0037] The hosts **101** are illustratively configured to write data to and read data from the distributed storage system **102** comprising storage nodes **105** in accordance with applications executing on those hosts **101** for system users.

[0038] The term “user” herein is intended to be broadly construed so as to encompass numerous arrangements of human, hardware, software or firmware entities, as well as combinations of such entities. Compute and/or storage services may be provided for users under a Platform-as-a-Service (PaaS) model, an Infrastructure-as-a-Service (IaaS) model and/or a Function-as-a-Service (FaaS) model, although it is to be appreciated that numerous other cloud infrastructure arrangements could be used. Also, illustrative embodiments can be implemented outside of the cloud infrastructure context, as in the case of a stand-alone computing and storage system implemented within a given enterprise. Combinations of cloud and edge infrastructure can also be used in implementing a given information processing system to provide services to users.

[0039] Communications between the components of system **100** can take place over additional or alternative networks, including a global computer network such as the Internet, a wide area network (WAN), a local area network (LAN), a satellite network, a telephone or cable network, a cellular network such as 4G or 5G cellular network, a wireless network such as a WiFi or WiMAX network, or various portions or combinations of these and other types of networks. The system **100** in some embodiments therefore comprises one or more additional networks other than network **104** each comprising processing devices configured to communicate using TCP, IP and/or other communication protocols.

[0040] As a more particular example, some embodiments may utilize one or more high-speed local networks in which associated processing devices communicate with one another utilizing Peripheral Component Interconnect express (PCIe) interface cards of those devices, that support networking protocols such as InfiniBand or Fibre Channel, in addition to or in place of TCP/IP. Numerous alternative networking arrangements are possible in a given embodiment, as will be appreciated by those skilled in the art. Additional examples include remote direct memory access (RDMA) over Converged Ethernet (RoCE) or RDMA over iWARP.

[0041] The first storage node **105-1** comprises a plurality of storage devices **106-1** and an associated storage processor **108-1**. The storage devices **106-1** illustratively store metadata pages and user data pages associated with one or more storage volumes of the distributed storage system **102**. The storage volumes illustratively comprise respective logical units (LUNs) or other types of logical storage volumes (e.g., NVMe namespaces). The storage devices **106-1** in some embodiments more particularly comprise local persistent storage devices of the first storage node **105-1**. Such persistent storage devices are local to the first storage node **105-1**, but remote from the second storage node **105-2**, the storage node **105-M** and any other ones of other storage nodes **105**.

[0042] Each of the other storage nodes **105-2** through **105-M** is assumed to be configured in a manner similar to that described above for the first storage node **105-1**. Accordingly, by way of example, storage node **105-2** comprises a plurality of storage devices **106-2** and an associated storage processor **108-2**, and storage node **105-M** comprises a plurality of storage devices **106-M** and an associated storage processor **108-M**.

[0043] As indicated previously, the storage devices **106-2** through **106-M** illustratively store metadata pages and user data pages associated with one or more storage volumes of the distributed storage system **102**, such as the above-noted LUNs or other types of logical storage volumes. The storage devices **106-2** in some embodiments more particularly comprise local persistent storage devices of the storage node **105-2**. Such persistent storage devices are local to the storage node **105-2**, but remote from the first storage node **105-1**, the storage node **105-M**, and any other ones of the storage nodes **105**. Similarly, the storage devices **106-M** in some embodiments more

particularly comprise local persistent storage devices of the storage node **105-M**. Such persistent storage devices are local to the storage node **105-M**, but remote from the first storage node **105-1**, the second storage node **105-2**, and any other ones of the storage nodes **105**.

[0044] The local persistent storage of a given one of the storage nodes **105** illustratively comprises the particular local persistent storage devices that are implemented in or otherwise associated with that storage node.

[0045] The storage processors **108** of the storage nodes **105** may include additional modules and other components typically found in conventional implementations of storage processors and storage systems, although such additional modules and other components are omitted from the figure for clarity and simplicity of illustration.

[0046] Additionally or alternatively, the storage processors **108** in some embodiments can comprise or be otherwise associated with one or more write caches and one or more write cache journals, both also illustratively distributed across the storage nodes **105** of the distributed storage system. It is further assumed in illustrative embodiments that one or more additional journals are provided in the distributed storage system, such as, for example, a metadata update journal and possibly other journals providing other types of journaling functionality for IO operations. Illustrative embodiments disclosed herein are assumed to be configured to perform various destaging processes for write caches and associated journals, and to perform additional or alternative functions in conjunction with processing of IO operations.

[0047] The storage devices **106** of the storage nodes **105** illustratively comprise solid state drives (SSDs). Such SSDs are implemented using non-volatile memory (NVM) devices such as flash memory. Other types of NVM devices that can be used to implement at least a portion of the storage devices **106** include, for example, non-volatile random access memory (NVRAM), phase-change RAM (PC-RAM), magnetic RAM (MRAM), resistive RAM, and spin torque transfer magneto-resistive RAM (STT-MRAM). These and various combinations of multiple different types of NVM devices may also be used. For example, hard disk drives (HDDs) can be used in combination with or in place of SSDs or other types of NVM devices.

[0048] However, it is to be appreciated that other types of storage devices can be used in other embodiments. For example, a given storage system as the term is broadly used herein can include a combination of different types of storage devices, as in the case of a multi-tier storage system comprising a flash-based fast tier and a disk-based capacity tier. In such an embodiment, each of the fast tier and the capacity tier of the multi-tier storage system comprises a plurality of storage devices with different types of storage devices being used in different ones of the storage tiers. For example, the fast tier may comprise flash drives while the capacity tier comprises HDDs. The particular storage devices used in a given storage tier may be varied in other embodiments, and multiple distinct storage device types may be used within a single storage tier. The term “storage device” as used herein is intended to be broadly construed, so as to encompass, for example, SSDs, HDDs, flash drives, hybrid drives or other types of storage devices. Such storage devices are examples of local persistent storage devices that may be used to implement at least a portion of the storage devices **106** of the storage nodes **105** of the distributed storage system of FIG. 1.

[0049] In some embodiments, the storage nodes **105** collectively provide a distributed storage system, although the storage nodes **105** can be used to implement other types of storage systems in other embodiments. One or more such storage nodes can be associated with at least one storage array. Additional or alternative types of storage products that can be used in implementing a given storage system in illustrative embodiments include software-defined storage, cloud storage and object-based storage. Combinations of multiple ones of these and other storage types can also be used.

[0050] As indicated above, the storage nodes **105** in some embodiments comprise respective software-defined storage server nodes of a software-defined storage system, in which the number and types of storage nodes **105** can be dynamically expanded or contracted under software control

using software-defined storage techniques.

[0051] The term “storage system” as used herein is therefore intended to be broadly construed, and should not be viewed as being limited to certain types of storage systems, such as content addressable storage systems or flash-based storage systems. A given storage system as the term is broadly used herein can comprise, for example, network-attached storage (NAS), storage area networks (SANs), direct-attached storage (DAS) and distributed DAS, as well as combinations of these and other storage types, including software-defined storage.

[0052] In some embodiments, communications between the hosts **101** and the storage nodes **105** comprise NVMe commands of an NVMe storage access protocol, for example, as described in the NVMe Specification, Revision 2.0c, October 2022, which is incorporated by reference herein. Other examples of NVMe storage access protocols that may be utilized in illustrative embodiments disclosed herein include NVMe over Fabrics, also referred to herein as NVMe-OF, and NVMe over TCP, also referred to herein as NVMe/TCP. Other embodiments can utilize other types of storage access protocols. As another example, communications between the hosts **101** and the storage nodes **105** in some embodiments can comprise Small Computer System Interface (SCSI) commands and the Internet SCSI (iSCSI) protocol.

[0053] Other types of commands may be used in other embodiments, including commands that are part of a standard command set, or custom commands such as a “vendor unique command” or VU command that is not part of a standard command set. The term “command” as used herein is therefore intended to be broadly construed, so as to encompass, for example, a composite command that comprises a combination of multiple individual commands. Numerous other types, formats and configurations of IO operations can be used in other embodiments, as that term is broadly used herein.

[0054] Some embodiments disclosed herein are configured to utilize one or more RAID arrangements to store data across the storage devices **106** in each of one or more of the storage nodes **105** of the distributed storage system **102**. Other embodiments can utilize other data protection techniques, such as, for example, Erasure Coding (EC), instead of one or more RAID arrangements.

[0055] The RAID arrangement can comprise, for example, a RAID 5 arrangement supporting recovery from a failure of a single one of the plurality of storage devices, a RAID 6 arrangement supporting recovery from simultaneous failure of up to two of the storage devices, or another type of RAID arrangement. For example, some embodiments can utilize RAID arrangements with redundancy higher than two.

[0056] The term “RAID arrangement” as used herein is intended to be broadly construed, and should not be viewed as limited to RAID 5, RAID 6 or other parity RAID arrangements. For example, a RAID arrangement in some embodiments can comprise combinations of multiple instances of distinct RAID approaches, such as a mixture of multiple distinct RAID types (e.g., RAID 1 and RAID 6) over the same set of storage devices, or a mixture of multiple stripe sets of different instances of one RAID type (e.g., two separate instances of RAID 5) over the same set of storage devices. Other types of parity RAID techniques and/or non-parity RAID techniques can be used in other embodiments.

[0057] Such a RAID arrangement is illustratively established by the storage processors **108** of the respective storage nodes **105**. The storage devices **106** in the context of RAID arrangements herein are also referred to as “disks” or “drives.” A given such RAID arrangement may also be referred to in some embodiments herein as a “RAID array.”

[0058] The RAID arrangement used in an illustrative embodiment includes a plurality of devices, each illustratively a different physical storage device of the storage devices **106**. Multiple such physical storage devices are typically utilized to store data of a given LUN or other logical storage volume in the distributed storage system. For example, data pages or other data blocks of a given LUN or other logical storage volume can be “striped” along with its corresponding parity

information across multiple ones of the devices in the RAID arrangement in accordance with RAID 5 or RAID 6 techniques.

[0059] A given RAID 5 arrangement defines block-level striping with single distributed parity and provides fault tolerance of a single drive failure, so that the array continues to operate with a single failed drive, irrespective of which drive fails. For example, in a conventional RAID 5 arrangement, each stripe includes multiple data blocks as well as a corresponding p parity block. The p parity blocks are associated with respective row parity information computed using well-known RAID 5 techniques. The data and parity blocks are distributed over the devices to support the above-noted single distributed parity and its associated fault tolerance.

[0060] A given RAID 6 arrangement defines block-level striping with double distributed parity and provides fault tolerance of up to two drive failures, so that the array continues to operate with up to two failed drives, irrespective of which two drives fail. For example, in a conventional RAID 6 arrangement, each stripe includes multiple data blocks as well as corresponding p and q parity blocks. The p and q parity blocks are associated with respective row parity information and diagonal parity information computed using well-known RAID 6 techniques. The data and parity blocks are distributed over the devices to collectively provide a diagonal-based configuration for the p and q parity information, so as to support the above-noted double distributed parity and its associated fault tolerance.

[0061] In such RAID arrangements, the parity blocks are typically not read unless needed for a rebuild process triggered by one or more storage device failures.

[0062] These and other references herein to RAID 5, RAID 6 and other particular RAID arrangements are only examples, and numerous other RAID arrangements can be used in other embodiments. Also, other embodiments can store data across the storage devices **106** of the storage nodes **105** without using RAID arrangements.

[0063] In some embodiments, the storage nodes **105** of the distributed storage system of FIG. **1** are connected to each other in a full mesh network, and are collectively managed by a system manager. A given set of local persistent storage devices or other storage devices **106** on a given one of the storage nodes **105** is illustratively implemented in a disk array enclosure (DAE) or other type of storage array enclosure of that storage node. Each of the storage nodes **105** illustratively comprises a CPU or other type of processor, a memory, a network interface card (NIC) or other type of network interface, and its corresponding storage devices **106**, possibly arranged as part of a DAE of the storage node.

[0064] In some embodiments, different ones of the storage nodes **105** are associated with the same DAE or other type of storage array enclosure. The system manager is illustratively implemented as a management module or other similar management logic instance, possibly running on one or more of the storage nodes **105**, on another storage node and/or on a separate non-storage node of the distributed storage system.

[0065] As a more particular non-limiting illustration, the storage nodes **105** in some embodiments are paired together in an arrangement referred to as a “brick,” with each such brick being coupled to a different DAE comprising multiple drives, and each node in a brick being connected to the DAE and to each drive through a separate connection. The system manager may be running on one of the two nodes of a first one of the bricks of the distributed storage system. Again, numerous other arrangements of the storage nodes are possible in a given distributed storage system as disclosed herein.

[0066] The system **100** as shown further comprises a plurality of system management nodes **110** that are illustratively configured to provide system management functionality of the type noted above. Such functionality in the present embodiment illustratively further involves utilization of control plane servers **112** and a system management database **116**. In some embodiments, at least portions of the system management nodes **110** and their associated control plane servers **112** are distributed over the storage nodes **105**. For example, a designated subset of the storage nodes **105**

can each be configured to include a corresponding one of the control plane servers **112**. Other system management functionality provided by system management nodes **110** can be similarly distributed over a subset of the storage nodes **105**.

[0067] The system management database **116** stores configuration and operation information of the system **100** and portions thereof are illustratively accessible to various system administrators such as host administrators and storage administrators.

[0068] The hosts **101-1**, **101-2**, . . . **101-N** include respective instances of path selection logic **114-1**, **114-2**, . . . **114-N**. Such instances of path selection logic **114** are illustratively utilized in supporting functionality for dynamic adjustment of network resources in the distributed storage system **102**, illustratively through interaction with IO queue adjustment logic instances implemented in respective ones of the storage processors **108** of the storage nodes **105**, as described in more detail below.

[0069] In some embodiments, each of the storage nodes **105** of the distributed storage system **102** is assumed to comprise multiple controllers associated with a corresponding target of that storage node. Such a “target” as that term is broadly used herein is illustratively a destination end of one or more paths from one or more of the hosts **101** to the storage node, and may comprise, for example, an NVMe subsystem of the storage node, although other types of targets can be used in other embodiments. It should be noted that different types of targets may be present in NVMe embodiments than are present in other embodiments that use other storage access protocols, such as SCSI embodiments. Accordingly, the types of targets that may be implemented in a given embodiment can vary depending upon the particular storage access protocol being utilized in that embodiment, and/or other factors. Similarly, the types of initiators can vary depending upon the particular storage access protocol, and/or other factors. Again, terms such as “initiator” and “target” as used herein are intended to be broadly construed, and should not be viewed as being limited in any way to particular types of components associated with any particular storage access protocol.

[0070] The paths that are selected by instances of path selection logic **114** of the hosts **101** for delivering IO operations from the hosts **101** to the distributed storage system **102** are associated with respective initiator-target pairs, as described in more detail elsewhere herein.

[0071] In some embodiments, IO operations are processed in the hosts **101** utilizing their respective instances of path selection logic **114** in the following manner. A given one of the hosts **101** establishes a plurality of paths between at least one initiator of the given host and a plurality of targets of respective storage nodes **105** of the distributed storage system **102**. For each of a plurality of IO operations generated in the given host for delivery to the distributed storage system **102**, the host selects a path to a particular target, and sends the IO operation to the corresponding storage node over the selected path.

[0072] The given host above is an example of what is more generally referred to herein as “at least one processing device” that includes a processor coupled to a memory. The storage nodes **105** of the distributed storage system **102** are also examples of “at least one processing device” as that term is broadly used herein.

[0073] It is to be appreciated that path selection as disclosed herein can be performed independently by each of the hosts **101**, illustratively utilizing their respective instances of path selection logic **114**, as indicated above, with possible involvement of additional or alternative system components.

[0074] In some embodiments, the initiator of the given host and the targets of the respective storage nodes **105** are configured to support one or more designated standard storage access protocols, such as an NVMe access protocol or a SCSI access protocol. As more particular examples in the NVMe context, the designated storage access protocol may comprise an NVMe/FC or NVMe/TCP access protocol, although a wide variety of additional or alternative storage access protocols can be used in other embodiments.

[0075] The hosts **101** can comprise additional or alternative components. For example, in some

embodiments, the hosts **101** further comprise respective sets of IO queues and respective multi-path input-output (MPIO) drivers. The MPIO drivers collectively comprise a multi-path layer of the hosts **101**. Path selection functionality for delivery of IO operations from the hosts **101** to the distributed storage system **102** is provided in the multi-path layer by respective instances of path selection logic implemented within the MPIO drivers. In some embodiments, the instances of path selection logic **114** are implemented at least in part within the MPIO drivers of the hosts **101**. [0076] The MPIO drivers may comprise, for example, otherwise conventional MPIO drivers, such as PowerPath® drivers from Dell Technologies, suitably modified in the manner disclosed herein to provide one or more portions of the disclosed functionality for dynamic adjustment of network resources. Other types of MPIO drivers from other driver vendors may be suitably modified to incorporate one or more portions of the functionality for dynamic adjustment of network resources as disclosed herein.

[0077] For example, the instances of path selection logic **114** of the respective hosts **101** can be implemented at least in part in respective MPIO drivers of those hosts.

[0078] In some embodiments, such instances of path selection logic **114** include or are otherwise associated with respective corresponding instances of host-side IO queue adjustment logic that are configured to send commands associated with dynamic network resource adjustment, such as, for example, a Set Features command to specify an initial number of IO queues to be configured for a particular TCP connection, to targets of the storage nodes **105** of the distributed storage system **102**.

[0079] Such host-side IO queue adjustment logic can be part of an MPIO layer of the hosts **101**, and possibly deployed at least in part within a corresponding instance of path selection logic **114** in the MPIO layer, or can be implemented elsewhere within the hosts **101**.

[0080] In some embodiments, the hosts **101** comprise respective local caches, implemented using respective memories of those hosts. A given such local cache can be implemented using one or more cache cards. A wide variety of different caching techniques can be used in other embodiments, as will be appreciated by those skilled in the art. Other examples of memories of the respective hosts **101** that may be utilized to provide local caches include one or more memory cards or other memory devices, such as, for example, an NVMe over PCIe cache card, a local flash drive or other type of NVM storage drive, or combinations of these and other host memory devices.

[0081] The MPIO drivers are illustratively configured to deliver IO operations selected from their respective sets of IO queues to the distributed storage system **102** via selected ones of multiple paths over the network **104**. The sources of the IO operations stored in the sets of IO queues illustratively include respective processes of one or more applications executing on the hosts **101**. For example, IO operations can be generated by each of multiple processes of a database application running on one or more of the hosts **101**. Such processes issue IO operations for delivery to the distributed storage system **102** over the network **104**. Other types of sources of IO operations may be present in a given implementation of system **100**.

[0082] A given IO operation is therefore illustratively generated by a process of an application running on a given one of the hosts **101**, and is queued in one of the IO queues of the given host with other operations generated by other processes of that application, and possibly other processes of other applications.

[0083] The paths from the given host to the distributed storage system **102** illustratively comprise paths associated with respective initiator-target pairs, with each initiator comprising, for example, a port of a single-port or multi-port host bus adaptor (HBA) or other initiating entity of the given host and each target comprising a port or other targeted entity corresponding to one or more of the storage devices **106** of the distributed storage system **102**. As noted above, the storage devices **106** illustratively comprise LUNs or other types of logical storage devices.

[0084] In some embodiments, the paths are associated with respective communication links between the given host and the distributed storage system **102** with each such communication link

having a negotiated link speed. For example, in conjunction with registration of a given HBA to a switch of the network **104**, the HBA and the switch may negotiate a link speed. The actual link speed that can be achieved in practice in some cases is less than the negotiated link speed, which is a theoretical maximum value.

[0085] Negotiated rates of the respective particular initiator and the corresponding target illustratively comprise respective negotiated data rates determined by execution of at least one link negotiation protocol for an associated one of the paths.

[0086] In some embodiments, at least a portion of the initiators comprise virtual initiators, such as, for example, respective ones of a plurality of N-Port ID Virtualization (NPIV) initiators associated with one or more Fibre Channel (FC) network connections. Such initiators illustratively utilize NVMe arrangements such as NVMe/FC, although other protocols can be used. Other embodiments can utilize other types of virtual initiators in which multiple network addresses can be supported by a single network interface, such as, for example, multiple media access control (MAC) addresses on a single network interface of an Ethernet network interface card (NIC). Accordingly, in some embodiments, the multiple virtual initiators are identified by respective ones of a plurality of media MAC addresses of a single network interface of a NIC. Such initiators illustratively utilize NVMe arrangements such as NVMe/TCP, although again other protocols can be used.

[0087] Accordingly, in some embodiments, multiple virtual initiators are associated with a single HBA of a given one of the hosts **101** but have respective unique identifiers associated therewith.

[0088] Additionally or alternatively, different ones of the multiple virtual initiators are illustratively associated with respective different ones of a plurality of virtual machines of the given host that share a single HBA of the given host, or a plurality of logical partitions of the given host that share a single HBA of the given host.

[0089] Numerous alternative virtual initiator arrangements are possible, as will be apparent to those skilled in the art. The term “virtual initiator” as used herein is therefore intended to be broadly construed. It is also to be appreciated that other embodiments need not utilize any virtual initiators. References herein to the term “initiators” are intended to be broadly construed, and should therefore be understood to encompass physical initiators, virtual initiators, or combinations of both physical and virtual initiators.

[0090] Various scheduling algorithms, load balancing algorithms and/or other types of algorithms can be utilized by the MPIO driver of the given host in delivering IO operations from the IO queues of that host to the distributed storage system **102** over particular paths via the network **104**. Each such IO operation is assumed to comprise one or more commands for instructing the distributed storage system **102** to perform particular types of storage-related functions such as reading data from or writing data to particular logical volumes of the distributed storage system **102**. Such commands are assumed to have various payload sizes associated therewith, and the payload associated with a given command is referred to herein as its “command payload.”

[0091] A command directed by the given host to the distributed storage system **102** is considered an “outstanding” command until such time as its execution is completed in the viewpoint of the given host, at which time it is considered a “completed” command. The commands illustratively comprise respective NVMe commands, although other command formats, such as SCSI command formats, can be used in other embodiments. In the SCSI context, a given such command is illustratively defined by a corresponding command descriptor block (CDB) or similar format construct. The given command can have multiple blocks of payload associated therewith, such as a particular number of 512-byte SCSI blocks or other types of blocks. Other command formats, e.g., Submission Queue Entry (SQE), are utilized in the NVMe context.

[0092] In illustrative embodiments to be described below, it is assumed without limitation that the initiators of a plurality of initiator-target pairs comprise respective ports of the given host and that the targets of the plurality of initiator-target pairs comprise respective ports of the distributed storage system **102**. Examples of such host ports and storage array ports are illustrated in

conjunction with the embodiment of FIG. 5. The host ports can comprise, for example, ports of single-port HBAs and/or ports of multi-port HBAs, or other types of host ports, including network interface cards (NICs). A wide variety of other types and arrangements of initiators and targets can be used in other embodiments.

[0093] Selecting a particular one of multiple available paths for delivery of a selected one of the IO operations from the given host is more generally referred to herein as “path selection.” Path selection as that term is broadly used herein can in some cases involve both selection of a particular IO operation and selection of one of multiple possible paths for accessing a corresponding logical device of the distributed storage system **102**. The corresponding logical device illustratively comprises a LUN or other logical storage volume to which the particular IO operation is directed.

[0094] It should be noted that paths may be added or deleted between the hosts **101** and the distributed storage system **102** in the system **100**. For example, the addition of one or more new paths from the given host to the distributed storage system **102** or the deletion of one or more existing paths from the given host to the distributed storage system **102** may result from respective addition or deletion of at least a portion of the storage devices **106** of the distributed storage system **102**.

[0095] Addition or deletion of paths can also occur as a result of zoning and masking changes or other types of storage system reconfigurations performed by a storage administrator or other user. Some embodiments are configured to send a predetermined command from the given host to the distributed storage system **102**, illustratively utilizing the MPIO driver, to determine if zoning and masking information has been changed. The predetermined command can comprise, for example, a log sense command, a mode sense command, a “vendor unique command” or VU command, or combinations of multiple instances of these or other commands, in an otherwise standardized command format.

[0096] In some embodiments, paths are added or deleted in conjunction with addition of a new storage array or deletion of an existing storage array from a storage system that includes multiple storage arrays, possibly in conjunction with configuration of the storage system for at least one of a migration operation and a replication operation.

[0097] For example, a storage system may include first and second storage arrays, with data being migrated from the first storage array to the second storage array prior to removing the first storage array from the storage system.

[0098] As another example, a storage system may include a production storage array and a recovery storage array, with data being replicated from the production storage array to the recovery storage array so as to be available for data recovery in the event of a failure involving the production storage array.

[0099] In these and other situations, path discovery scans may be repeated as needed in order to discover the addition of new paths or the deletion of existing paths.

[0100] A given path discovery scan can be performed utilizing known functionality of conventional MPIO drivers, such as PowerPath® drivers.

[0101] The path discovery scan in some embodiments may be further configured to identify one or more new LUNs or other logical storage volumes associated with the one or more new paths identified in the path discovery scan. The path discovery scan may comprise, for example, one or more bus scans which are configured to discover the appearance of any new LUNs that have been added to the distributed storage system **102** as well to discover the disappearance of any existing LUNs that have been deleted from the distributed storage system **102**.

[0102] The MPIO driver of the given host in some embodiments comprises a user-space portion and a kernel-space portion. The kernel-space portion of the MPIO driver may be configured to detect one or more path changes of the type mentioned above, and to instruct the user-space portion of the MPIO driver to run a path discovery scan responsive to the detected path changes. Other divisions of functionality between the user-space portion and the kernel-space portion of the MPIO

driver are possible. The user-space portion of the MPIO driver is illustratively associated with an Operating System (OS) kernel of the given host.

[0103] For each of one or more new paths identified in the path discovery scan, the given host may be configured to execute a host registration operation for that path. The host registration operation for a given new path illustratively provides notification to the distributed storage system **102** that the given host has discovered the new path.

[0104] As indicated previously, the storage nodes **105** of the distributed storage system **102** process IO operations from one or more hosts **101** and in processing those IO operations run various storage application processes that generally involve interaction of that storage node with one or more other ones of the storage nodes.

[0105] In the FIG. **1** embodiment, the distributed storage system **102** comprises storage processors **108** and corresponding sets of storage devices **106**, and may include additional or alternative components, such as sets of local caches.

[0106] The storage processors **108** illustratively control the processing of IO operations received in the distributed storage system **102** from the hosts **101**. For example, the storage processors **108** illustratively manage the processing of read and write commands directed by the MPIO drivers of the hosts **101** to particular ones of the storage devices **106**. The storage processors **108** can be implemented as respective storage controllers, directors or other storage system components configured to control storage system operations relating to processing of IO operations. In some embodiments, each of the storage processors **108** has a different one of the above-noted local caches associated therewith, although numerous alternative arrangements are possible.

[0107] The manner in which functionality for dynamic adjustment of network resources is implemented in system **100** will now be described in more detail.

[0108] As indicated previously, in software-defined storage system arrangements utilizing advanced storage access protocols such as NVMe-oF or NVMe/TCP, it can be unduly difficult to configure an appropriate level of network resources for targets of respective storage nodes. For example, conventional approaches are susceptible to overallocation of network resources, which can undermine storage system performance.

[0109] Illustrative embodiments disclosed herein provide techniques for dynamic adjustment of network resources, in a software-defined storage system or other type of storage system, such as numbers of IO queues and associated TCP connections configured for each of one or more NVMe targets of the storage system. Such techniques advantageously facilitate the usage of advanced storage access protocols such as NVMe-oF or NVMe/TCP while avoiding the above-described drawbacks of conventional practice. For example, some embodiments can substantially improve storage system performance by dynamically adjusting network resources, such as the number of IO queues and corresponding TCP connections for NVMe targets of the storage system, responsive to actual usage of the network resources over time.

[0110] In some embodiments, an NVMe target comprises one or more NVMe controllers, each having a set of TCP connections associated with an Admin queue and one or more IO queues. Each TCP connection illustratively corresponds to a single queue having associated request/response entries. In some embodiments, the TCP connections corresponding to a given Admin queue and a set of one or more IO queues are collectively referred to as a “TCP association.” The IO queues and their corresponding TCP connections are examples of what are more generally referred to herein as “network resources.” Such network resources in some embodiments are illustratively used to receive IO operations directed to targets of a storage system from initiators of one or more host devices. Additional or alternative network resources can be used in other embodiments.

[0111] Typically, a fixed number of IO queues are established, with a corresponding fixed number of TCP connections, in accordance with maximum load requirements of one or more applications that will be directing IO operations to the NVMe target for processing. Unfortunately, this approach often results in the overprovisioning of resources such as IO queues and their corresponding TCP

connections, such that many allocated IO queues and their corresponding active TCP connections remain unused. This is particularly problematic in highly-scaled distributed storage systems with large numbers of storage nodes. For example, the resulting excessively large numbers of overprovisioned resources can lead to unstable behavior when the storage system is running out of resources.

[0112] These and other problems are addressed in illustrative embodiments by configuring an initial relatively small number of IO queues and TCP connections for a given NVMe target, and then dynamically adjusting the number of IO queues and corresponding TCP connections, based at least in part on the actual IO load received for the previously-configured relatively small number of IO queues and TCP connections. Such an arrangement helps to conserve network resources and improves overall storage system performance, particularly in highly-scaled storage system deployments.

[0113] Accordingly, illustrative embodiments disclosed herein monitor and dynamically increase the number of IO queues and corresponding TCP connections for a given NVMe target, based at least in part on actual usage of previously-allocated numbers of IO queues and TCP connections. Such increases may be triggered by monitored IO queue utilization exceeding a specified threshold amount of IO queue utilization. The numbers of IO queues and their corresponding TCP connections for a given NVMe target are therefore initially configured at relatively low levels, and subsequently increased responsive to actual usage.

[0114] Such an approach advantageously avoids complications that might otherwise be associated with alternative approaches such as increasing TCP window size and/or increasing send/receive buffer sizes for TCP sockets. For example, such alternative approaches can require administrator rights, can adversely impact other application flows and/or can be restricted by kernel computations based on available system memory.

[0115] In some embodiments, when an NVMe initiator connects to an NVMe target, the NVMe initiator sends a Set Features command to the NVMe target, with the Set Features command including a Feature Identifier that specifies an initial number of IO queues to be configured, where the initial number of IO queues is a relatively low number of IO queues, substantially less than a maximum available number of IO queues. It is assumed in some embodiments that each IO queue corresponds to a separate TCP connection, although other arrangements are possible in other embodiments.

[0116] In some embodiments, the NVMe target or other storage system component or set of components is configured to periodically or otherwise monitor states of the IO queues in terms of IO queue utilization, and to add one or more IO queues as necessary responsive to a specified threshold IO queue utilization level being exceeded. For example, monitoring may be performed every t seconds, and for a given monitoring instance, if the determined IO queue utilization level exceeds a threshold limit of L percent of full utilization, one or more additional IO queues are configured for the NVMe target. By way of example, in some embodiments, the threshold limit of L percent of full utilization, for IO queue utilization as measured collectively across all presently-configured IO queues, may be about 90% and the period of time for monitoring may be about 1 to 10 seconds, although other values can be used. In some embodiments, the initial number of IO queues is a minimum number of IO queues given by 16 queues, and the maximum number of IO queues is given by 128 queues, although other values could be used. Again, in some embodiments, it is assumed that each IO queue has a corresponding TCP connection, although other arrangements are possible.

[0117] It is to be noted in this regard that the NVMe/TCP specification allows the addition of new IO queues, after an initial setup of IO queues, as long as total number of IO queues and their associated queue identifiers do not exceed maximum values previously agreed upon at setup by the initiator and the target. For example, in some embodiments, a host device can request a maximum number of IO queues at setup, and the target can agree to that maximum number or provide a lower

maximum number. This is illustratively done during an initial setup when an Admin queue is established for the target. After this setup, the host can send the above-noted Set Features command to the target to specify an initial number of IO queues to be configured, where the initial number of IO queues is a relatively low number of IO queues, substantially less than the maximum number of IO queues previously agreed upon. Accordingly, the term “initial number of IO queues” as used herein is intended to be broadly construed, so as to encompass, for example, a particular number of IO queues to be configured, in some cases after a maximum number of IO queues has already been established. In such embodiments, it is illustratively up to the host device to specify the initial number of IO queues to be configured.

[0118] In some embodiments, the number of IO queues is not subsequently decreased after an increase in the number of IO queues. Such an arrangement assumes that if a given application requires increased throughput at a given point in time, it may subsequently require similarly increased throughput at a future point in time, and so reductions in the number of configured IO queues and corresponding TCP connections are not made. However, other embodiments can be configured to dynamically adjust numbers of IO queues by both increasing and decreasing the number of configured IO queues based on actual usage as needed based on results of the periodic queue fullness monitoring.

[0119] As mentioned above, each of the storage nodes **105** of the distributed storage system **102** illustratively comprises one or more targets, where each such target is associated with multiple distinct paths from respective HBAs or other initiators of one or more of the hosts **101**. Illustrative embodiments disclosed herein configure the distributed storage system **102** to dynamically adjust network resources associated with network connections between initiators of the one or more of the hosts **101** and targets of the storage nodes **105**. Such network resources illustratively comprise IO queues and corresponding TCP connections for each of a plurality of targets of the storage nodes **105**.

[0120] For example, in some embodiments, one or more of the storage nodes **105** each implements at least one target, such as an NVMe target, that is configured to include multiple controllers, such as at least a first controller associated with a first storage pool, and a second controller associated with a second storage pool. The first and second storage pools are illustratively storage pools of the distributed storage system **102**, and such storage pools may be distributed across multiple ones of the storage nodes **105**. Each of the first and second storage pools is assumed to comprise one or more LUNs or other logical storage volumes.

[0121] Although first and second controllers are referred to in conjunction with some embodiments herein, it is to be appreciated that more than two controllers can be implemented in a given target in order to support more than two storage pools.

[0122] A given one of the storage nodes **105** illustratively processes IO operations received from one or more of the hosts **101**, with different ones of the IO operations being directed by the one or more hosts **101** from one or more initiators of the one or more hosts **101** to different ones of the first and second controllers of the target implemented within the given storage node.

[0123] The dynamic adjustment of network resources in such an embodiment is illustratively implemented in the following manner.

[0124] The distributed storage system **102** in an illustrative embodiment is configured to receive from a given one of the hosts **101** a command specifying an initial number of IO queues to be configured for a given target of the distributed storage system **102**, to configure in the distributed storage system **102** the initial number of IO queues for the given target responsive to the received command, and to dynamically adjust in the distributed storage system **102** the number of IO queues configured for the given target based at least in part on an amount of IO operations received by the distributed storage system **102** for the given target.

[0125] This process in illustrative embodiments is more particularly performed by each of the storage nodes **105** of the distributed storage system **102**, for each of a plurality of targets

implemented within that storage node.

[0126] In some embodiments, each of the IO queues configured in the distributed storage system **102** for the given target is associated with a corresponding different TCP connection between the given host and the given target.

[0127] The given target may comprise at least one NVMe controller of a particular one of the storage nodes **105** of the distributed storage system **102**, although other types of targets can be used.

[0128] In some embodiments, the command comprises a Set Features command having a Feature Identifier specifying the initial number of IO queues for the given target and is received in the given target of the distributed storage system **102** from an initiator of the given host. Other types of commands can be used in other embodiments to establish the initial number of IO queues to be configured for the given target in the distributed storage system **102**.

[0129] In some embodiments, dynamically adjusting in the distributed storage system **102** the number of IO queues configured for the given target based at least in part on an amount of IO operations received by the distributed storage system **102** for the given target illustratively comprises increasing the number of IO queues relative to the initial number of IO queues based at least in part on a utilization level of the IO queues. The utilization level of the IO queues is illustratively determined collectively across all currently-configured IO queues for the given target for each of a plurality of monitoring intervals.

[0130] In some embodiments, increasing the number of IO queues relative to the initial number of IO queues based at least in part on the utilization level of the IO queues comprises adding one or more IO queues and respective associated TCP connections to a total number of IO queues and respective associated TCP connections previously configured for the given target, responsive to the utilization level of the IO queues exceeding a designated threshold utilization level. The designated threshold utilization level may be expressed at least in part as a particular percentage of full utilization of the IO queues for the given target.

[0131] Additionally or alternatively, dynamically adjusting in the distributed storage system **102** the number of IO queues configured for the given target based at least in part on an amount of IO operations received by the distributed storage system **102** for the given target comprises varying the number of IO queues configured for the given target within a range from the initial number of IO queues for the given target to a specified maximum number of IO queues for the given target. The specified maximum number of IO queues for the given target may be separately agreed upon by the given host and the given target, illustratively prior to the sending of the above-noted Set Features command, utilizing another Set Features command with a Feature Identifier specifying the maximum number of IO queues for the given target. Again, other types of commands can be used in other embodiments.

[0132] Such dynamic adjustment of the number of IO queues as disclosed herein can in some embodiments include adjusting additional or alternative characteristics of one or more IO queues, such as the size of one or more of the IO queues and/or the types of one or more of the IO queues. Terms such as “dynamic adjustment” as used herein in the context of the number of IO queues are therefore intended to be broadly construed, so as to encompass modifications to various types of queue-related parameters, such as queue size, queue type, etc.

[0133] As indicated above, in some embodiments disclosed herein, a given one of the hosts **101** illustratively generates a command, such as a Set Features command, to specify an initial number of IO queues for a given TCP connection as disclosed herein, and sends it over a selected path to the given storage node. In some embodiments, a corresponding target of the given storage node receives the command from an initiator of the given host. The command illustratively includes a Feature Identifier specifying the initial number of IO queues to be configured in the storage node for the corresponding TCP connection.

[0134] The given storage node configures the initial number of IO queues for the target responsive

to the received command.

[0135] The term “target” as used herein in the context of a distributed storage system or other type of storage system is intended to be broadly construed.

[0136] The target in some embodiments more particularly comprises multiple controllers accessible via respective different associations comprising one or more TCP connections between the given host and the given storage node. For example, the target may comprise a plurality of NVMe controllers of an NVMe subsystem of the given storage node.

[0137] The other storage nodes **105** are each assumed to be configured in a manner similar to that described above and elsewhere herein for the given storage node.

[0138] As indicated above, in some embodiments, multiple controllers are part of a single physical controller subsystem of the given storage node. For example, first and second controllers may comprise respective NVMe controllers of an NVMe subsystem of the given storage node. Such an NVMe subsystem is considered an example of what is more generally referred to herein as a “target” of the given storage node.

[0139] An example of such an arrangement will be described in more detail below in conjunction with FIG. 4. Other types of targets comprising multiple controllers supporting dynamic adjustment of network resources can be used in other embodiments.

[0140] The first and second controllers in some embodiments may be viewed as comprising respective “virtual” controllers associated with the single physical controller subsystem of the given storage node.

[0141] Additionally or alternatively, the first and second controllers in some embodiments are accessible via respective first and second different associations comprising one or more TCP connections between a given one of the one or more hosts **101** and the given storage node. In such an arrangement, a host accesses the first controller using the first association, and accesses the second controller using the second association. Such associations are also referred to herein as TCP associations, and may include, for each of at least one Admin queue and a plurality of IO queues, a corresponding TCP connection. Other types of communication links can be used in other embodiments.

[0142] In some embodiments, the first controller comprises a first set of IO queues and the second controller comprises a second set of IO queues, for use in processing IO operations for their respective storage pools. Again, each IO queue in a given such set of IO queues may be associated with a separate TCP connection over which a given one of the hosts **101** communicates with the corresponding controller.

[0143] An additional example of an illustrative process for implementing at least some of the above-described dynamic resource adjustment functionality will be provided below in conjunction with the flow diagram of FIG. 2.

[0144] As indicated previously, the storage nodes **105** collectively comprise an example of a distributed storage system. The term “distributed storage system” as used herein is intended to be broadly construed, so as to encompass, for example, scale-out storage systems, clustered storage systems or other types of storage systems distributed over multiple storage nodes.

[0145] Also, the term “storage volume” as used herein is intended to be broadly construed, and should not be viewed as being limited to any particular format or configuration.

[0146] In some embodiments, the storage nodes **105** are implemented using processing modules that are interconnected in a full mesh network, such that a process of one of the processing modules can communicate with processes of any of the other processing modules. Commands issued by the processes can include, for example, remote procedure calls (RPCs) directed to other ones of the processes.

[0147] The sets of processing modules of the storage nodes **105** illustratively comprise control modules, data modules, routing modules and at least one management module. Again, these and possibly other processing modules of the storage nodes **105** are illustratively interconnected with

one another in the full mesh network, such that each of the modules can communicate with each of the other modules, although other types of networks and different module interconnection arrangements can be used in other embodiments.

[0148] The management module in such an embodiment may more particularly comprise a system-wide management module, also referred to herein as a system manager. Other embodiments can include multiple instances of the management module implemented on different ones of the storage nodes **105**.

[0149] A wide variety of alternative configurations of nodes and processing modules are possible in other embodiments. Also, the term “storage node” as used herein is intended to be broadly construed, and may comprise a node that implements storage control functionality but does not necessarily incorporate storage devices. As mentioned previously, a given storage node can in some embodiments comprise a separate storage array, or a portion of a storage array that includes multiple such storage nodes.

[0150] Communication links may be established between the various processing modules of the storage nodes using well-known communication protocols such as TCP/IP and ROCE. For example, respective sets of IP links used in data transfer and corresponding messaging could be associated with respective different ones of the routing modules.

[0151] The particular features described above in conjunction with FIG. **1** should not be construed as limiting in any way, and a wide variety of other system arrangements implementing dynamic adjustment of network resources as disclosed herein are possible.

[0152] The storage nodes **105** of the example distributed storage system **102** illustrated in FIG. **1** are assumed to be implemented using at least one processing platform, with each such processing platform comprising one or more processing devices, and each such processing device comprising a processor coupled to a memory. Such processing devices can illustratively include particular arrangements of compute, storage and network resources.

[0153] The storage nodes **105** may be implemented on respective distinct processing platforms, although numerous other arrangements are possible. At least portions of their associated hosts **101** may be implemented on the same processing platforms as the storage nodes **105** or on separate processing platforms.

[0154] The term “processing platform” as used herein is intended to be broadly construed so as to encompass, by way of illustration and without limitation, multiple sets of processing devices and associated storage systems that are configured to communicate over one or more networks. For example, distributed implementations of the system **100** are possible, in which certain components of the system reside in one data center in a first geographic location while other components of the system reside in one or more other data centers in one or more other geographic locations that are potentially remote from the first geographic location. Thus, it is possible in some implementations of the system **100** for different subsets of the hosts **101** and the storage nodes **105** to reside in different data centers. Numerous other distributed implementations of the storage nodes **105** and their respective associated sets of hosts **101** are possible.

[0155] Additional examples of processing platforms utilized to implement storage systems and possibly their associated hosts in illustrative embodiments will be described in more detail below in conjunction with FIGS. **6** and **7**.

[0156] It is to be appreciated that these and other features of illustrative embodiments are presented by way of example only, and should not be construed as limiting in any way.

[0157] Accordingly, different numbers, types and arrangements of system components such as hosts **101**, distributed storage system **102**, storage nodes **105**, storage devices **106**, storage processors **108**, system management nodes **110**, control plane servers **112** and instances of path selection logic **114** can be used in other embodiments. For example, as mentioned previously, system management functionality of system management nodes **110** can be distributed across a subset of the storage nodes **105**, instead of being implemented on separate nodes.

[0158] It should therefore be understood that the particular sets of modules and other components implemented in a distributed storage system as illustrated in FIG. 1 are presented by way of example only. In other embodiments, only subsets of these components, or additional or alternative sets of components, may be used, and such components may exhibit alternative functionality and configurations.

[0159] For example, in other embodiments, certain portions of dynamic resource adjustment functionality as disclosed herein can be implemented in one or more hosts, in a storage system, or partially in a host and partially in a storage system. Accordingly, illustrative embodiments are not limited to arrangements in which dynamic resource adjustment functionality is implemented primarily in storage system or primarily in a particular host or set of hosts, and therefore such embodiments encompass various alternative arrangements, such as, for example, an arrangement in which the functionality is distributed over one or more storage systems and one or more associated hosts, each comprising one or more processing devices. The term “at least one processing device” as used herein is therefore intended to be broadly construed.

[0160] The operation of the information processing system **100** will now be described in further detail with reference to the flow diagram of the illustrative embodiment of FIG. 2, which illustrates a process for dynamic adjustment of network resources as disclosed herein. This process may be viewed as an example algorithm implemented at least in part by distributed storage system **102** interacting with one or more of the hosts **101**. These and other algorithms for dynamic adjustment of network resources as disclosed herein can be implemented using other types and arrangements of system components in other embodiments.

[0161] The process illustrated in FIG. 2 includes steps **200** through **204**, and in some implementations may be performed primarily by at least a given one of the storage nodes **105** of the distributed storage system **102** interacting with a given one of the hosts **101**. Similar processes may be performed primarily by other ones of the storage nodes **105** interacting with the given host or other ones of the hosts **101**, although it is to be appreciated that other implementations are possible in other embodiments.

[0162] In step **200**, a storage system receives from a given host a command specifying an initial number of IO queues to be configured for a given target of the storage system. The term “target” as used herein is intended to be broadly construed, and in some embodiments can comprise, for example, an NVMe subsystem, which is more generally referred to herein as an NVMe target. The NVMe subsystem or other NVMe target in such an arrangement illustratively comprises one or more controllers. An example illustrating multiple controllers of a single physical NVMe subsystem of the storage system will be described in more detail below in conjunction with the illustrative embodiment of FIG. 4. In some embodiments, the given target is implemented on a particular storage node, which receives from an initiator of the given host a Set Features command to set an initial number of IO queues for the given target as disclosed herein. The Set Features command illustratively provides a Feature Identifier that specifies an initial number of IO queues to be configured for the given target in the storage system.

[0163] In step **202**, the initial number of IO queues for the given target is configured in the storage system responsive to the received command. For example, the initial number of IO queues and respective corresponding TCP associations are configured for the given target. The given target and/or other storage system components can be utilized in configuring the initial number of IO queues and respective corresponding TCP associations. In some embodiments, each IO queue has its own separate TCP connection, although other arrangements can be used. The set of TCP connections utilized for respective ones of the IO queues and possibly also an Admin queue of the given target are collectively referred to herein as a “TCP association.” The IO queues and their associated TCP connections are examples of what are more generally referred to herein as “network resources.”

[0164] In step **204**, the number of IO queues configured in the storage system for the given target is

dynamically adjusted based at least in part on an amount of IO operations received by the storage system for the given target.

[0165] For example, dynamically adjusting the number of IO queues configured for the given target may comprise increasing the number of IO queues relative to the initial number of IO queues based at least in part on a utilization level of the IO queues. The utilization level of the IO queues may be determined collectively across all currently-configured IO queues for the given target for each of a plurality of monitoring intervals. Increasing the number of IO queues relative to the initial number of IO queues based at least in part on the utilization level of the IO queues may more particularly comprise adding one or more IO queues and respective associated TCP connections to a total number of IO queues and respective associated TCP connections previously configured for the given target, responsive to the utilization level of the IO queues exceeding a designated threshold utilization level. The designated threshold utilization level may be expressed, for example, at least in part as a particular percentage of full utilization of the IO queues for the given target. In some embodiments, dynamically adjusting the number of IO queues configured for the given target based at least in part on an amount of IO operations received by the storage system for the given target illustratively comprises varying the number of IO queues configured for the given target within a range from the initial number of IO queues for the given target to a specified maximum number of IO queues for the given target.

[0166] One or more of steps **200** through **204** are illustratively repeated over time in order to support the dynamic resource adjustment functionality. Multiple such processes may operate in parallel with one another in order to provide dynamic network resource adjustment functionality for different targets and their corresponding respective storage nodes and for different initiators and their respective corresponding hosts.

[0167] The steps of the FIG. 2 process are shown in sequential order for clarity and simplicity of illustration only, and certain steps can at least partially overlap with other steps. Additional or alternative steps can be used in other embodiments.

[0168] The particular processing operations and other system functionality described in conjunction with the flow diagram of FIG. 2 are therefore presented by way of illustrative example only, and should not be construed as limiting the scope of the disclosure in any way. Alternative embodiments can use other types of processing operations for implementing dynamic adjustment of network resources for one or more hosts and a storage system. For example, as indicated above, the ordering of the process steps may be varied in other embodiments, or certain steps may be performed at least in part concurrently with one another rather than serially. Also, one or more of the process steps may be repeated periodically, or multiple instances of the process can be performed in parallel with one another in order to implement a plurality of different processes for respective different hosts and/or storage nodes.

[0169] Functionality such as that described in conjunction with the flow diagram of FIG. 2 can be implemented at least in part in the form of one or more software programs stored in memory and executed by a processor of a processing device such as a computer or server. As will be described below, a memory or other storage device having executable program code of one or more software programs embodied therein is an example of what is more generally referred to herein as a “processor-readable storage medium.”

[0170] One or more hosts and/or one or more storage nodes can be implemented as part of what is more generally referred to herein as a processing platform comprising one or more processing devices each comprising a processor coupled to a memory.

[0171] A given such processing device in some embodiments may correspond to one or more virtual machines or other types of virtualization infrastructure such as Docker containers or Linux containers (LXCs). Hosts, storage processors and other system components may be implemented at least in part using processing devices of such processing platforms. For example, respective path selection logic instances and other related logic instances of the hosts can be implemented in

respective containers running on respective ones of the processing devices of a processing platform.

[0172] Additional examples of illustrative embodiments will now be described with reference to FIGS. 3 through 5. As mentioned previously, FIG. 3 comprises two figures denoted FIG. 3A and FIG. 3B.

[0173] Referring initially to FIG. 3A, this embodiment illustrates an example of a distributed storage system that more particularly comprises a software-defined storage system having a plurality of software-defined storage server nodes, also referred to as SDS server nodes, configured to utilize an NVMe storage access protocol such as NVMe-oF or NVMe/TCP. Such SDS server nodes are examples of “storage nodes” as that term is broadly used herein. As will be appreciated by those skilled in the art, similar embodiments can be implemented without the use of software-defined storage and with other storage access protocols.

[0174] As shown in FIG. 3A, an information processing system **300** comprises a host **301** configured to communicate over a network **304**, illustratively a TCP/IP network, with a software-defined storage system comprising a plurality of SDS server nodes **305-1**, **305-2**, . . . **305-M** and corresponding control plane servers **312**. The control plane servers **312** are shown in dashed outline as the functionality of such servers in illustrative embodiments is distributed over a particular subset of the SDS server nodes **305** rather than being implemented on separate nodes of the software-defined storage system. The control plane servers **312** provide system management functionality such as centralized storage provisioning, monitoring, membership management, as well as storage partitioning.

[0175] A plurality of applications **311** execute on the host **301** and generate IO operations that are delivered to particular ones of the SDS server nodes **305** via at least one NVMe initiator **318**. The host **301** further comprises path selection logic **314** and IO queue adjustment logic **315**, illustratively configured to carry out aspects of dynamic resource adjustment functionality of the host **301** in a manner similar to that previously described. In other embodiments, the IO queue adjustment logic **315** may be part of the path selection logic **314**, rather than a separate component as illustrated in the figure. Both the path selection logic **314** and the IO queue adjustment logic **315** in some embodiments are implemented at least in part within an MPIO driver of the host **301**. Although only a single host **301** is shown in system **300**, the system **300** can include multiple hosts, each configured as generally shown for host **301**, as in the system **100** of FIG. 1.

[0176] Each of the SDS server nodes **305** in the present embodiment comprises at least one NVMe target **320**, a data relay agent **321**, a data server **322** and a set of local drives **323**. The internal components of a given SDS server node with the exception of the local drives **323** are illustratively part of a corresponding storage processor in the FIG. 1 embodiment, although numerous other arrangements are possible.

[0177] The data relay agent **321** facilitates relaying of IO requests between different ones of the SDS server nodes **305**, and the data servers **322** provide access to data stored in the local drives **323** of their respective SDS server nodes **305**. Additional or alternative components may be included in the SDS server nodes **305** in illustrative embodiments.

[0178] Although single NVMe initiators and targets are shown in respective ones of the host **301** and the SDS server nodes **305**, this is by way of simplified illustration only, and other embodiments can include multiple NVMe initiators within host **301** and multiple NVMe targets within each of the SDS server nodes **305**. Also, one or more instances of IO queue adjustment logic are assumed to be implemented within or otherwise associated with each of the NVMe targets, although such storage-side logic instances are not shown in the figure.

[0179] In some embodiments, the SDS server nodes **305** are configured at least in part as respective PowerFlex® software-defined storage nodes from Dell Technologies, suitably modified as disclosed herein to implement dynamic adjustment of network resources, although other types of storage nodes can be used in other embodiments.

[0180] The NVMe targets **320** in some embodiments collectively comprise an NVMe subsystem that implements multiple distinct controllers and associated aspects of the dynamic adjustment of network resources. For example, a given such NVMe target can comprise at least a first controller associated with a first storage pool of the distributed storage system, and a second controller associated with a second storage pool of the distributed storage system. Other types and arrangements of multiple controllers can be used.

[0181] A given one of the SDS server nodes **305** processes IO operations received from the host **301**, with different ones of the IO operations being directed by the host **301** from NVMe initiator **318** to different ones of the first and second controllers of the NVMe target **320** of the given SDS server node.

[0182] The IO queue adjustment logic **315** of the host **301** illustratively controls the generation of commands relating to dynamic adjustment of network resources as disclosed herein. Such commands include the previously-described Set Features command to specify an initial number of IO queues to be configured for a particular TCP connection by the NVMe target **320** of the given SDS server node.

[0183] In some embodiments, a Set Features command is utilized to establish an initial number of IO queues as disclosed herein. For example, when an NVMe initiator connects to an NVMe target, the NVMe initiator sends to the NVMe target a Set Features command which illustratively conveys a Feature Identifier that specifies a particular initial number of IO queues to be configured for the NVMe target. Each such IO queue illustratively has an associated TCP connection. The NVMe target illustratively sends a corresponding reply confirming the particular initial number of IO queues to be configured for that target. The initial number of IO queues in some embodiments is a relatively small number of IO queues, substantially less than a maximum number of IO queues that might otherwise be established.

[0184] An example Set Features command includes an operation code (“Op code”) denoted 0x9, a zeroes portion, a Command Identifier, a number of command data word (“Dword”) instances, including Dword 1 through Dword 15, a Save(S) bit, a Universally Unique Identifier (UUID) index, and a Feature Identifier (FID), the latter illustratively specifying an initial number of IO queues to be configured for a given target. In this embodiment, one or more command data words can be used to convey the specified initial number of IO queues for the given target. For example, a particular one of the command data words, or a designated portion thereof, can be used to convey the specified initial number of IO queues for the given target.

[0185] This Set Features command will cause the receiving NVMe target **320** to configure the initial number of IO queues. The NVMe target **320** will then dynamically adjust the number of IO queues in the manner disclosed herein.

[0186] The Command Identifier, the S bit and the UUID Index are not Features specific but are part of the NVMe specification. The S bit may be cleared to zero for this vendor-specific NVMe feature, and the UUID Index is set if supported by the controller, in accordance with the NVMe specification.

[0187] Other types and arrangements of commands, fields and storage access protocols can be used in other embodiments.

[0188] It is to be appreciated that the particular command formats described above are presented by way of illustrative example only, and numerous other formats can be used in other embodiments. In implementing such command functionality, one or more logic instances or other components/drivers of the host **301** can be modified to support the particular command formats being utilized. For example, one or more MPIO drivers or other multi-path drivers of the host **301** may be modified as disclosed herein. Additionally or alternatively, other host drivers, such as one or more host NVMe-over-TCP drivers, can be modified to support the functionality disclosed herein. The SDS server nodes **305** are also assumed to be configured to support these example command formats, or other similar formats, in illustrative embodiments.

[0189] Referring now to FIG. 3B, example pseudocode is shown for dynamic adjustment of network resources in the system **300**.

[0190] As indicated previously, in some embodiments, the NVMe target or one or more other storage system components or sets of components are configured to periodically or otherwise monitor states of the IO queues in terms of IO queue utilization, and to add one or more IO queues as necessary responsive to a specified threshold IO queue utilization level being exceeded. For example, as described elsewhere herein, monitoring may be performed every *t* seconds, and for a given monitoring instance, if the determined IO queue utilization level exceeds a threshold limit of *L* percent of full utilization, one or more additional IO queues are configured for the NVMe target. As a more particular example, the threshold limit of *L* percent of full utilization, for IO queue utilization as measured collectively across all presently-configured IO queues, may be about 90% and the period of time for monitoring may be about 1 to 10 seconds, although other values can be used.

[0191] The example pseudocode of FIG. 3B implements this type of dynamic adjustment of network resources. It is assumed in this example that the set of IO queues associated with a given target each include respective transmit and receive portions, collectively providing a transmit buffer size and a receive buffer size, although a wide variety of other IO queue arrangements can be used in other embodiments. The example pseudocode computes, illustratively over the set of IO queues associated with the given target, a percentage utilization of transmit queue length (“txQueueLen”) relative to a total transmit buffer size (“tx_buf_size”) and compares it to the threshold limit of *L* percent, and similarly computes a percentage utilization of receive queue length (“rxQueueLen”) relative to a total receive buffer size (“rx_buf_size”) and also compares it to the threshold limit of *L* percent. If at least one of the computation results exceeds the threshold limit of *L* percent, a new IO queue is added, as indicated. These operations are repeated for each of a plurality of monitoring intervals. It is to be appreciated that this is only an example, and numerous other types of dynamic adjustment mechanisms may be implemented in other embodiments.

[0192] FIG. 4 illustrates an example association between different controllers and respective storage pools in one embodiment.

[0193] In the FIG. 4 embodiment, an information processing system **400** comprises a host **401** that communicates with a storage system **402** via a plurality of TCP associations including at least a first TCP association **403-1** and a second TCP association **403-2**, also denoted as TCP Association 1 and TCP Association 2, respectively.

[0194] A given such “TCP association” as that term is broadly used herein illustratively comprises one or more TCP connections of an association between a host and a controller. For example, an association between a host and a controller may comprise multiple TCP connections, one for an Admin Queue and one for each of a plurality of IO queues. A TCP association and its one or more TCP connections are each assumed to be examples of what are more generally referred to herein as “network connections.” The term “network connection” as used herein is therefore intended to be broadly construed. IO queues of the controllers are illustratively associated with respective ones of the TCP connections, and the IO queues and their respective TCP connections illustratively represent examples of what are more generally referred to herein as “network resources” subject to dynamic adjustment.

[0195] The storage system **402** may comprise, for example, a software-defined storage system or other type of distributed storage system, or a non-distributed storage system. Although only a single host **401** is shown in the figure, the system **400** can include multiple hosts.

[0196] The storage system **402** comprises an NVMe subsystem **420**. The NVMe subsystem **420** is an example of what is more generally referred to herein as a “target” of the storage system **402**, and although only a single such target is shown in the figure, the storage system **402** can include multiple such targets, each illustratively implemented as a separate NVMe subsystem.

[0197] The NVMe subsystem **420** in this embodiment comprises a first controller **430-1** and a

second controller **430-2**, also denoted as Controller **1** and Controller **2**, respectively. The first controller **430-1** is associated with a first storage pool **SP1** of the storage system **402**. The second controller **430-2** is associated with a second storage pool **SP2** of the storage system **402**. In other embodiments, the NVMe subsystem **420** can comprise more than two controllers **430**, each associated with one or more storage pools. Accordingly, other embodiments can include only a single storage pool, or can involve different associations between multiple controllers and multiple storage pools. The NVMe subsystem **420** in the present embodiment is illustratively a single physical controller subsystem of the storage system **402**, and the first and second controllers **430-1** and **430-2** may be viewed as comprising respective virtual controllers associated with that single physical controller subsystem, although numerous other arrangements are possible in other embodiments.

[0198] The storage system **402** processes IO operations received from the host **401** via the TCP associations **403**. The host **401** directs IO operations from an NVMe initiator to the first controller **430-1** via the first TCP association **403-1**, and directs IO operations from the NVMe initiator to the second controller **430-2** via the second TCP association **403-2**.

[0199] As illustrated in the figure, the first controller **430-1** comprises a first set of IO queues denoted IO Queue **1** through IO Queue **n**, and the second controller **430-2** comprises a second set of IO queues denoted IO Queue **1** through IO Queue **m**, where **m** may be greater than, equal to or less than **n**. The capacities of the first and second sets of IO queues of the first and second controllers **430-1** and **430-2** may be made different from one another by configuring those first and second sets of IO queues to have at least one of a different number of IO queues and a different IO queue size relative to one another, although in other embodiments the first and second sets of IO queues may have the same capacity.

[0200] In the FIG. **4** embodiment, the NVMe target comprising NVMe subsystem **420** may be configured as a software-defined target or SDT of a software-defined storage system implementation of storage system **402**. Such a target can be implemented at least in part as a Linux user space component, illustratively comprising a daemon listening for incoming TCP connections from one or more NVMe initiators of the host **401**. The NVMe target comprising NVMe subsystem **420** is illustratively accessible from the host **401** via the TCP associations **403**, each associated with a different one of the first and second controllers **430-1** and **430-2** of the NVMe subsystem **420**.

[0201] In accordance with the above-noted dynamic adjustment functionality, an initial number of IO queues is established for each of the controllers **430**, and then dynamically adjusted based on actual usage of those IO queues, using the techniques disclosed elsewhere herein. It is to be appreciated that the particular configuration of controllers **430** and their respective sets of IO queues shown in FIG. **4** is presented by way of illustrative example only, and can be varied in other embodiments.

[0202] Referring now to FIG. **5**, another illustrative embodiment is shown. In this embodiment, an information processing system **500** comprises host-side elements that include application processes **511**, path selection logic **514** and IO queue adjustment logic **515**, and storage-side elements that include IO queues per target **521** and IO queue adjustment logic **522**. The path selection logic **514** is configured to operate in conjunction with IO queue adjustment logic **515**, IO queues per target **521** and IO queue adjustment logic **522** to implement functionality for dynamic adjustment of network resources in the system **500**. There may be separate instances of one or more such elements associated with each of a plurality of system components such as hosts and storage arrays of the system **500**. For example, different instances of the path selection logic **514** and IO queue adjustment logic **515** are illustratively implemented within or otherwise in association with respective ones of a plurality of MPIO drivers of respective hosts. In other embodiments, the IO queue adjustment logic **515** can be implemented at least in part within the path selection logic **514**.

[0203] The system **500** is configured in accordance with a layered system architecture that illustratively includes a host processor layer **530**, an MPIO layer **532**, a host port layer **534**, a

switch fabric layer **536**, a storage array port layer **538** and a storage array processor layer **540**. The host processor layer **530**, the MPIO layer **532** and the host port layer **534** are associated with one or more hosts, the switch fabric layer **536** is associated with one or more SANs or other types of networks, and the storage array port layer **538** and storage array processor layer **540** are associated with one or more storage arrays (“SAs”). A given such storage array illustratively comprises a software-defined storage system or other type of distributed storage system comprising a plurality of storage nodes.

[0204] In a manner similar to that described elsewhere herein, one or more storage arrays of the system **500** are each configured to implement at least one storage-side target that includes multiple controllers, such as, for example, at least a first controller associated with a first storage pool, and a second controller associated with a second storage pool, where the first and second controllers each include respective sets of IO queues that are dynamically adjusted in number based at least in part on actual usage thereof in the manner described elsewhere herein.

[0205] Accordingly, the one or more storage arrays of system **500** implement dynamic adjustment of network resources provided by targets of the one or more storage arrays to initiators of one or more hosts. For example, Set Features commands to specify initial numbers of IO queues for respective controllers or other targets are illustratively generated by or otherwise under the control of the host-side IO queue adjustment logic **515** and sent over paths selected by the path selection logic **514** to the targets of the one or more storage arrays. Corresponding IO queues per target **521** are established by the one or more storage arrays for respective ones of the targets the initial numbers of IO queues specified in respective Set Features commands. Dynamic adjustment of the numbers of IO queues and their associated TCP connections is then implemented under the control of the IO queue adjustment logic **522** in a manner that dynamically varies a number of the IO queues per target **521** for each of one or more of the targets based at least in part on actual utilization of the corresponding IO queues.

[0206] The system **500** in this embodiment therefore implements dynamic resource adjustment functionality utilizing one or more MPIO drivers of the MPIO layer **532**, and associated instances of path selection logic **514** and IO queue adjustment logic **515**, as well as the IO queues per target **521** and the IO queue adjustment logic **522**.

[0207] The one or more storage arrays also process IO operations received from one or more hosts, with different ones of the IO operations being directed by the one or more hosts under the control of path selection logic **514** from one or more initiators of the one or more hosts to different ones of the first and second controllers of the target in a given storage array.

[0208] For example, in some embodiments, a given one of the hosts of the system **500** is illustratively configured to establish an initial number of IO queues to be used in each of one or more targets (e.g., NVME targets) of respective storage nodes. For each of a plurality of IO operations generated by one or more of the application processes **511** in the given host for delivery to the given storage array, the given host selects, illustratively via path selection logic **514** of one or more MPIO drivers of the MPIO layer **532**, a particular one of the plurality of paths from the initiator to one of the targets on the particular storage node, and sends the IO operation to the particular storage node over the selected path.

[0209] The application processes **511** generate IO operations that are processed by the MPIO layer **532** for delivery to the one or more storage arrays that collectively comprise a plurality of storage nodes of a distributed storage system. Paths are determined by the path selection logic **514** for sending such IO operations to the one or more storage arrays. These IO operations are sent to the one or more storage arrays in accordance with one or more scheduling algorithms, load balancing algorithms and/or other types of algorithms.

[0210] The MPIO layer **532** is an example of what is also referred to herein as a multi-path layer, and comprises one or more MPIO drivers implemented in respective hosts. Each such MPIO driver illustratively comprises respective instances of path selection logic **514** and IO queue adjustment

logic **515** configured as previously described. Additional or alternative layers and logic arrangements can be used in other embodiments.

[0211] The IO queues per target **521** implemented in the storage array processor layer **540** illustratively comprise, for respective targets, a plurality of IO queues that starts from an initial relatively small number of IO queues specified by a given host device, and varies over time in accordance with the techniques disclosed herein, in a range between the initial relatively small number of IO queues and a maximum number of IO queues.

[0212] As mentioned above, in the system **500**, path selection logic **514** is configured to select different paths for sending IO operations from a given host to a storage array. These paths as illustrated in the figure include a first path from a particular host port denoted **HP1** through a particular switch fabric denoted **SF1** to a particular storage array port denoted **SP1**, and a second path from another particular host port denoted **HP2** through another particular switch fabric denoted **SF2** to another particular storage array port denoted **SP2**.

[0213] These two particular paths are shown by way of illustrative example only, and in many practical implementations there will typically be a much larger number of paths between the one or more hosts and the one or more storage arrays, depending upon the specific system configuration and its deployed numbers of host ports, switch fabrics and storage array ports. For example, each host in the FIG. 5 embodiment can illustratively have the same number and type of paths to a shared storage array, or alternatively different ones of the hosts can have different numbers and types of paths to the storage array.

[0214] The path selection logic **514** of the MPIO layer **532** in this embodiment selects paths for delivery of IO operations to the one or more storage arrays having the storage array ports of the storage array port layer **538**. More particularly, the path selection logic **514** determines appropriate paths over which to send particular IO operations to particular logical storage devices of the one or more storage arrays.

[0215] Some implementations of the system **500** can include a relatively large number of hosts (e.g., 1000 or more hosts), although as indicated previously different numbers of hosts, and possibly only a single host, may be present in other embodiments. Each of the hosts is typically allocated a sufficient number of host ports to accommodate predicted performance needs. In some cases, the number of ports per host is on the order of 4, 8 or 16, although other numbers of ports could be allocated to each host depending upon the predicted performance needs. A typical storage array may include on the order of 128 ports, although again other numbers can be used based on the particular needs of the implementation. The number of hosts per storage array port in some cases can be on the order of 10 hosts per port.

[0216] A given host of system **500** can be configured to initiate an automated path discovery process to discover new paths responsive to updated zoning and masking or other types of storage system reconfigurations performed by a storage administrator or other user. For certain types of hosts, such as hosts using particular operating systems such as Windows, ESX or Linux, automated path discovery via the MPIO drivers of a multi-path layer is typically supported. Other types of hosts using other operating systems such as AIX in some implementations do not necessarily support such automated path discovery, in which case alternative techniques can be used to discover paths.

[0217] These and other features of illustrative embodiments disclosed herein are examples only, and should not be construed as limiting in any way. Other types of dynamic adjustment of network resources can be used in other embodiments, and the term “dynamic adjustment” as used herein is intended to be broadly construed.

[0218] The above-described illustrative embodiments can provide significant advantages over conventional approaches.

[0219] For example, some embodiments provide techniques for dynamic adjustment of network resources, in a software-defined storage system or other type of distributed storage system,

illustratively utilizing initial numbers of IO queues that are specified for respective NVMe targets or other types of targets by one or more host devices. The storage system in some embodiments illustratively configures the specified initial number of IO queues for a given target, and then dynamically varies the number of IO queues configured for the given target over time based at least in part on actual usage of those IO queues in servicing IO operations received over the corresponding TCP connections.

[0220] Such embodiments better match the allocated numbers of IO queues and TCP connections to actual usage of the IO queues and TCP connections, thereby avoiding overallocation of network resources and improving storage system performance, particularly in implementations using advanced storage access protocols such as NVMe-oF or NVMe/TCP.

[0221] Furthermore, some embodiments can be implemented without requiring any change in the NVMe specification or other storage access protocol specification.

[0222] It is to be appreciated that the particular advantages described above and elsewhere herein are associated with particular illustrative embodiments and need not be present in other embodiments. Also, the particular types of information processing system features and functionality as illustrated in the drawings and described above are exemplary only, and numerous other arrangements may be used in other embodiments.

[0223] Illustrative embodiments of processing platforms utilized to implement hosts and distributed storage systems with dynamic resource adjustment functionality will now be described in greater detail with reference to FIGS. 6 and 7. Although described in the context of system 100, these platforms may also be used to implement at least portions of other information processing systems in other embodiments.

[0224] FIG. 6 shows an example processing platform comprising cloud infrastructure 600. The cloud infrastructure 600 comprises a combination of physical and virtual processing resources that may be utilized to implement at least a portion of the information processing system 100. The cloud infrastructure 600 comprises multiple virtual machines (VMs) and/or container sets 602-1, 602-2, . . . 602-L implemented using virtualization infrastructure 604. The virtualization infrastructure 604 runs on physical infrastructure 605, and illustratively comprises one or more hypervisors and/or operating system level virtualization infrastructure. The operating system level virtualization infrastructure illustratively comprises kernel control groups of a Linux operating system or other type of operating system.

[0225] The cloud infrastructure 600 further comprises sets of applications 610-1, 610-2, . . . 610-L running on respective ones of the VMs/container sets 602-1, 602-2, . . . 602-L under the control of the virtualization infrastructure 604. The VMs/container sets 602 may comprise respective VMs, respective sets of one or more containers, or respective sets of one or more containers running in VMs.

[0226] In some implementations of the FIG. 6 embodiment, the VMs/container sets 602 comprise respective VMs implemented using virtualization infrastructure 604 that comprises at least one hypervisor. Such implementations can provide dynamic resource adjustment functionality in a distributed storage system of the type described above using one or more processes running on a given one of the VMs. For example, each of the VMs can implement logic instances and/or other components for implementing functionality associated with dynamic adjustment of network resources in the system 100.

[0227] A hypervisor platform may be used to implement a hypervisor within the virtualization infrastructure 604. Such a hypervisor platform may comprise an associated virtual infrastructure management system. The underlying physical machines may comprise one or more distributed processing platforms that include one or more storage systems.

[0228] In other implementations of the FIG. 6 embodiment, the VMs/container sets 602 comprise respective containers implemented using virtualization infrastructure 604 that provides operating system level virtualization functionality, such as support for Docker containers running on bare

metal hosts, or Docker containers running on VMs. The containers are illustratively implemented using respective kernel control groups of the operating system. Such implementations can also provide dynamic resource adjustment functionality in a distributed storage system of the type described above. For example, a container host supporting multiple containers of one or more container sets can implement logic instances and/or other components for implementing functionality associated with dynamic adjustment of network resources in the system **100**.

[0229] As is apparent from the above, one or more of the processing devices or other components of system **100** may each run on a computer, server, storage device or other processing platform element. A given such element may be viewed as an example of what is more generally referred to herein as a “processing device.” The cloud infrastructure **600** shown in FIG. **6** may represent at least a portion of one processing platform. Another example of such a processing platform is processing platform **700** shown in FIG. **7**.

[0230] The processing platform **700** in this embodiment comprises a portion of system **100** and includes a plurality of processing devices, denoted **702-1**, **702-2**, **702-3**, . . . **702-K**, which communicate with one another over a network **704**.

[0231] The network **704** may comprise any type of network, including by way of example a global computer network such as the Internet, a WAN, a LAN, a satellite network, a telephone or cable network, a cellular network, a wireless network such as a WiFi or WiMAX network, or various portions or combinations of these and other types of networks.

[0232] The processing device **702-1** in the processing platform **700** comprises a processor **710** coupled to a memory **712**.

[0233] The processor **710** may comprise a microprocessor, a microcontroller, an application-specific integrated circuit (ASIC), a field-programmable gate array (FPGA), graphics processing unit (GPU) or other type of processing circuitry, as well as portions or combinations of such circuitry elements.

[0234] The memory **712** may comprise random access memory (RAM), read-only memory (ROM), flash memory or other types of memory, in any combination. The memory **712** and other memories disclosed herein should be viewed as illustrative examples of what are more generally referred to as “processor-readable storage media” storing executable program code of one or more software programs.

[0235] Articles of manufacture comprising such processor-readable storage media are considered illustrative embodiments. A given such article of manufacture may comprise, for example, a storage array, a storage disk or an integrated circuit containing RAM, ROM, flash memory or other electronic memory, or any of a wide variety of other types of computer program products. The term “article of manufacture” as used herein should be understood to exclude transitory, propagating signals. Numerous other types of computer program products comprising processor-readable storage media can be used.

[0236] Also included in the processing device **702-1** is network interface circuitry **714**, which is used to interface the processing device with the network **704** and other system components, and may comprise conventional transceivers.

[0237] The other processing devices **702** of the processing platform **700** are assumed to be configured in a manner similar to that shown for processing device **702-1** in the figure.

[0238] Again, the particular processing platform **700** shown in the figure is presented by way of example only, and system **100** may include additional or alternative processing platforms, as well as numerous distinct processing platforms in any combination, with each such platform comprising one or more computers, servers, storage devices or other processing devices.

[0239] For example, other processing platforms used to implement illustrative embodiments can comprise various arrangements of converged infrastructure.

[0240] It should therefore be understood that in other embodiments different arrangements of additional or alternative elements may be used. At least a subset of these elements may be

collectively implemented on a common processing platform, or each such element may be implemented on a separate processing platform.

[0241] As indicated previously, components of an information processing system as disclosed herein can be implemented at least in part in the form of one or more software programs stored in memory and executed by a processor of a processing device. For example, at least portions of the dynamic resource adjustment functionality provided by one or more components of a storage system as disclosed herein are illustratively implemented in the form of software running on one or more processing devices.

[0242] It should again be emphasized that the above-described embodiments are presented for purposes of illustration only. Many variations and other alternative embodiments may be used. For example, the disclosed techniques are applicable to a wide variety of other types of information processing systems, hosts, storage systems, storage nodes, storage devices, storage processors, initiators, targets, path selection logic instances, IO queue adjustment logic instances and other components. Also, the particular configurations of system and device elements and associated processing operations illustratively shown in the drawings can be varied in other embodiments. Moreover, the various assumptions made above in the course of describing the illustrative embodiments should also be viewed as exemplary rather than as requirements or limitations of the disclosure. Numerous other alternative embodiments within the scope of the appended claims will be readily apparent to those skilled in the art.

Claims

1. An apparatus comprising: at least one processing device comprising a processor coupled to a memory; the at least one processing device being configured: to receive in a storage system from a host device a command specifying an initial number of input-output (IO) queues to be configured for a given target of the storage system; to configure in the storage system the initial number of IO queues for the given target responsive to the received command; and to dynamically adjust in the storage system the number of IO queues configured for the given target based at least in part on an amount of IO operations received by the storage system for the given target.
2. The apparatus of claim 1 wherein each of the IO queues configured in the storage system for the given target is associated with a corresponding different Transmission Control Protocol (TCP) connection between the host device and the target.
3. The apparatus of claim 1 wherein the storage system comprises a distributed storage system that includes a plurality of storage nodes.
4. The apparatus of claim 3 wherein the distributed storage system comprises a software-defined storage system and the storage nodes comprise respective software-defined storage server nodes of the software-defined storage system.
5. The apparatus of claim 1 wherein the given target comprises at least one Non-Volatile Memory Express (NVMe) controller of the storage system.
6. The apparatus of claim 1 wherein the command comprises a Set Features command having a Feature Identifier specifying the initial number of IO queues for the given target and is received in the given target of the storage system from an initiator of the host device.
7. The apparatus of claim 1 wherein dynamically adjusting in the storage system the number of IO queues configured for the given target based at least in part on an amount of IO operations received by the storage system for the given target comprises increasing the number of IO queues relative to the initial number of IO queues based at least in part on a utilization level of the IO queues.
8. The apparatus of claim 7 wherein the utilization level of the IO queues is determined collectively across all currently-configured IO queues for the given target for each of a plurality of monitoring intervals.
9. The apparatus of claim 7 wherein increasing the number of IO queues relative to the initial

number of IO queues based at least in part on the utilization level of the IO queues comprises adding one or more IO queues and respective associated TCP connections to a total number of IO queues and respective associated TCP connections previously configured for the given target, responsive to the utilization level of the IO queues exceeding a designated threshold utilization level.

- 10.** The apparatus of claim 9 wherein the designated threshold utilization level is expressed at least in part as a particular percentage of full utilization of the IO queues for the given target.
- 11.** The apparatus of claim 1 wherein dynamically adjusting in the storage system the number of IO queues configured for the given target based at least in part on an amount of IO operations received by the storage system for the given target comprises varying the number of IO queues configured for the given target within a range from the initial number of IO queues for the given target to a specified maximum number of IO queues for the given target.
- 12.** A computer program product comprising a non-transitory processor-readable storage medium having stored therein program code of one or more software programs, wherein the program code when executed by at least one processing device comprising a processor coupled to a memory, causes the at least one processing device: to receive in a storage system from a host device a command specifying an initial number of input-output (IO) queues to be configured for a given target of the storage system; to configure in the storage system the initial number of IO queues for the given target responsive to the received command; and to dynamically adjust in the storage system the number of IO queues configured for the given target based at least in part on an amount of IO operations received by the storage system for the given target.
- 13.** The computer program product of claim 12 wherein each of the IO queues configured in the storage system for the given target is associated with a corresponding different Transmission Control Protocol (TCP) connection between the host device and the target.
- 14.** The computer program product of claim 12 wherein dynamically adjusting in the storage system the number of IO queues configured for the given target based at least in part on an amount of IO operations received by the storage system for the given target comprises increasing the number of IO queues relative to the initial number of IO queues based at least in part on a utilization level of the IO queues.
- 15.** The computer program product of claim 14 wherein the utilization level of the IO queues is determined collectively across all currently-configured IO queues for the given target for each of a plurality of monitoring intervals.
- 16.** A method comprising: receiving in a storage system from a host device a command specifying an initial number of input-output (IO) queues to be configured for a given target of the storage system; configuring in the storage system the initial number of IO queues for the given target responsive to the received command; and dynamically adjusting in the storage system the number of IO queues configured for the given target based at least in part on an amount of IO operations received by the storage system for the given target; wherein the method is performed by at least one processing device comprising a processor coupled to a memory.
- 17.** The method of claim 16 wherein each of the IO queues configured in the storage system for the given target is associated with a corresponding different Transmission Control Protocol (TCP) connection between the host device and the target.
- 18.** The method of claim 16 wherein dynamically adjusting in the storage system the number of IO queues configured for the given target based at least in part on an amount of IO operations received by the storage system for the given target comprises increasing the number of IO queues relative to the initial number of IO queues based at least in part on a utilization level of the IO queues.
- 19.** The method of claim 18 wherein the utilization level of the IO queues is determined collectively across all currently-configured IO queues for the given target for each of a plurality of monitoring intervals.
- 20.** The method of claim 18 wherein increasing the number of IO queues relative to the initial

number of IO queues based at least in part on the utilization level of the IO queues comprises adding one or more IO queues and respective associated TCP connections to a total number of IO queues and respective associated TCP connections previously configured for the given target, responsive to the utilization level of the IO queues exceeding a designated threshold utilization level.
